

Activity_ Course 7 Salifort Motors project lab

September 6, 2025

1 Capstone project: Providing data-driven suggestions for HR

1.1 Description and deliverables

This capstone project is an opportunity for you to analyze a dataset and build predictive models that can provide insights to the Human Resources (HR) department of a large consulting firm.

Upon completion, you will have two artifacts that you would be able to present to future employers. One is a brief one-page summary of this project that you would present to external stakeholders as the data professional in Salifort Motors. The other is a complete code notebook provided here. Please consider your prior course work and select one way to achieve this given project question. Either use a regression model or machine learning model to predict whether or not an employee will leave the company. The exemplar following this activity shows both approaches, but you only need to do one.

In your deliverables, you will include the model evaluation (and interpretation if applicable), a data visualization(s) of your choice that is directly related to the question you ask, ethical considerations, and the resources you used to troubleshoot and find answers or solutions.

2 PACE stages

2.1 Pace: Plan

Consider the questions in your PACE Strategy Document to reflect on the Plan stage.

In this stage, consider the following:

2.1.1 Understand the business scenario and problem

The HR department at Salifort Motors wants to take some initiatives to improve employee satisfaction levels at the company. They collected data from employees, but now they don't know what to do with it. They refer to you as a data analytics professional and ask you to provide data-driven suggestions based on your understanding of the data. They have the following question: what's likely to make the employee leave the company?

Your goals in this project are to analyze the data collected by the HR department and to build a model that predicts whether or not an employee will leave the company.

If you can predict employees likely to quit, it might be possible to identify factors that contribute to their leaving. Because it is time-consuming and expensive to find, interview, and hire new employees, increasing employee retention will be beneficial to the company.

2.1.2 Familiarize yourself with the HR dataset

The dataset that you'll be using in this lab contains 15,000 rows and 10 columns for the variables listed below.

Note: you don't need to download any data to complete this lab. For more information about the data, refer to its source on [Kaggle](#).

Variable	Description
satisfaction_level	Employee-reported job satisfaction level [0–1]
last_evaluation	Score of employee's last performance review [0–1]
number_project	Number of projects employee contributes to
average_monthly_hours	Average number of hours employee worked per month
time_spend_company	How long the employee has been with the company (years)
Work_accident	Whether or not the employee experienced an accident while at work
left	Whether or not the employee left the company
promotion_last_5years	Whether or not the employee was promoted in the last 5 years
Department	The employee's department
salary	The employee's salary (U.S. dollars)

Reflect on these questions as you complete the plan stage.

- Who are your stakeholders for this project?

Stakeholders for this project include internal stakeholders like employees, HR department and executive leadership, external stakeholders such as customers, suppliers and regulators.

Employees: The primary focus of this project is the employees as they provided the data and they will also benefit from potential improvements.

Human Resources(HR): This project will help the HR department better understand the data collected about the employees. In turn helping the department with managing the workforce and improving employee development.

Executive leadership: This project will help leadership teams and decision makers to improve business performance, increase profitability and operational efficiency.

- What are you trying to solve or accomplish?

The goal of this project is to better understand employee behaviour through applying advanced analytics techniques. Salifort's Senior leadership is concerned about employee turnover, and hope to better understand this behaviour. Salifort strives to create a corporate culture that supports employee success and professional development. Predicting if a employee will leave the company, and the reason for their departure, they could better understand the problem and develop a solution.

- What are your initial observations when you explore the data?

My initial observation focused on two key variables, data quality and data distribution. First, I checked for missing values, which could indicate incomplete survey responses or data entry errors. I also examined the datatypes of each column to ensure they are appropriate. Although no missing values were observed, there were 3008 duplicate rows. This seemed unlikely to be real entries so the duplicate rows were dropped.

I then looked at data distribution in regards to `tenure` and determined the outlier threshold. The number of rows containing outliers were counted and it seems like 824 entries lie above the 5.5 year threshold. Meaning that there are 824 employees that have been at Salifort Motors for more than five and a half years. The dataset seems to be skewed to the left with most employees working at the company for 3-4 years.

- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)

The resources I relied on for this part of the project was:

1. Pandas Documentation: Essential for data manipulation and analysis. I used it for understanding methods like `df.info()`, `df.describe()`, `df.rename()` etc.
 - <https://pandas.pydata.org/docs/reference/index.html>
2. Matplotlib and seaborn documentation: essential for creating data visualizations to understand the data's distribution and relationship. I used them for creating histograms, box plots and scatterplots.
 - <https://matplotlib.org/stable/api/index.html>
 - <https://seaborn.pydata.org/api.html>
3. Stack Overflow: I used this resource to find real world examples on how to solve specific problems.
 - <https://stackoverflow.com/>
 - Do you have any ethical considerations at this stage?

This stage has significant ethical considerations, primarily related to employee privacy and anonymity. The primary consideration is to ensure that no individuals can be identified from their data. If the dataset contains any unique identifiers, it should be removed or anonymized. Secondly, my observation should be partial and it should not introduce my own biases. I should avoid making assumptions and present the data objectively. Lastly, care must be taken to misinterpret the data's findings. I must be careful not to imply causation. Data exploration provides clues, not definite answers.

2.2 Step 1. Imports

- Import packages
- Load dataset

2.2.1 Import packages

```
[1]: # Import packages
import pandas as pd
import numpy as np

# Import packages for data visualization
import matplotlib.pyplot as plt
import seaborn as sns

# for displaying all of the columns in dataprames
pd.set_option('display.max_columns', None)
# Import packages for data modeling
from xgboost import XGBClassifier
from xgboost import XGBRegressor
from xgboost import plot_importance

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# For metrics and helpful functions
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score,\
f1_score, confusion_matrix, ConfusionMatrixDisplay, classification_report
from sklearn.metrics import roc_auc_score, roc_curve
from sklearn.tree import plot_tree

# For saving models
import pickle
```

2.2.2 Load dataset

Pandas is used to read a dataset called **HR_capstone_dataset.csv**. As shown in this cell, the dataset has been automatically loaded in for you. You do not need to download the .csv file, or provide more code, in order to access the dataset and proceed with this lab. Please continue with this activity by completing the following instructions.

```
[2]: # RUN THIS CELL TO IMPORT YOUR DATA.
```

```
# Load dataset into a dataframe
### YOUR CODE HERE ###
df0 = pd.read_csv("HR_capstone_dataset.csv")

# Display first few rows of the dataframe
df0.head()
```

```
[2]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

	time_spend_company	Work_accident	left	promotion_last_5years	Department	\
0	3	0	1	0	sales	
1	6	0	1	0	sales	
2	4	0	1	0	sales	
3	5	0	1	0	sales	
4	3	0	1	0	sales	

	salary
0	low
1	medium
2	medium
3	low
4	low

2.3 Step 2. Data Exploration (Initial EDA and data cleaning)

- Understand your variables
- Clean your dataset (missing data, redundant data, outliers)

2.3.1 Gather basic information about the data

```
[3]: # Gather basic information about the data
df0.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14999 entries, 0 to 14998
Data columns (total 10 columns):
#   Column                Non-Null Count  Dtype
---  -
0   satisfaction_level      14999 non-null  float64
1   last_evaluation         14999 non-null  float64
```

```

2  number_project      14999 non-null  int64
3  average_monthly_hours  14999 non-null  int64
4  time_spend_company  14999 non-null  int64
5  Work_accident      14999 non-null  int64
6  left              14999 non-null  int64
7  promotion_last_5years  14999 non-null  int64
8  Department         14999 non-null  object
9  salary            14999 non-null  object
dtypes: float64(2), int64(6), object(2)
memory usage: 1.1+ MB

```

2.3.2 Gather descriptive statistics about the data

```
[4]: # Gather descriptive statistics about the data
df0.describe()
```

```
[4]:
```

	satisfaction_level	last_evaluation	number_project \		
count	14999.000000	14999.000000	14999.000000		
mean	0.612834	0.716102	3.803054		
std	0.248631	0.171169	1.232592		
min	0.090000	0.360000	2.000000		
25%	0.440000	0.560000	3.000000		
50%	0.640000	0.720000	4.000000		
75%	0.820000	0.870000	5.000000		
max	1.000000	1.000000	7.000000		

	average_monthly_hours	time_spend_company	Work_accident	left \	
count	14999.000000	14999.000000	14999.000000	14999.000000	
mean	201.050337	3.498233	0.144610	0.238083	
std	49.943099	1.460136	0.351719	0.425924	
min	96.000000	2.000000	0.000000	0.000000	
25%	156.000000	3.000000	0.000000	0.000000	
50%	200.000000	3.000000	0.000000	0.000000	
75%	245.000000	4.000000	0.000000	0.000000	
max	310.000000	10.000000	1.000000	1.000000	

	promotion_last_5years
count	14999.000000
mean	0.021268
std	0.144281
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

2.3.3 Rename columns

As a data cleaning step, rename the columns as needed. Standardize the column names so that they are all in `snake_case`, correct any column names that are misspelled, and make column names more concise as needed.

```
[5]: # Display all column names
df0.columns
```

```
[5]: Index(['satisfaction_level', 'last_evaluation', 'number_project',
          'average_monthly_hours', 'time_spend_company', 'Work_accident', 'left',
          'promotion_last_5years', 'Department', 'salary'],
          dtype='object')
```

```
[6]: # Rename columns as needed

new_columns = [col.lower().replace(' ', '_') for col in df0.columns]
df0.columns = new_columns

df0 = df0.rename(columns={'time_spend_company': 'tenure',
                          'average_monthly_hours': 'average_monthly_hours'})

# Display all column names after the update
df0.columns
```

```
[6]: Index(['satisfaction_level', 'last_evaluation', 'number_project',
          'average_monthly_hours', 'tenure', 'work_accident', 'left',
          'promotion_last_5years', 'department', 'salary'],
          dtype='object')
```

2.3.4 Check missing values

Check for any missing values in the data.

```
[7]: # Check for missing values
df0.isna().sum()
```

```
[7]: satisfaction_level      0
     last_evaluation        0
     number_project         0
     average_monthly_hours  0
     tenure                 0
     work_accident          0
     left                   0
     promotion_last_5years  0
     department             0
     salary                 0
```

dtype: int64

There are no missing values in the data.

2.3.5 Check duplicates

Check for any duplicate entries in the data.

```
[8]: # Check for duplicates
df = pd.DataFrame(df0)
df.duplicated().sum()
```

```
[8]: 3008
```

3,008 rows contain duplicates. That is 20% of the data.

```
[9]: # Inspect some rows containing duplicates as needed
df[df.duplicated()].head()
```

```
[9]:
```

	satisfaction_level	last_evaluation	number_project	\
396	0.46	0.57	2	
866	0.41	0.46	2	
1317	0.37	0.51	2	
1368	0.41	0.52	2	
1461	0.42	0.53	2	

	average_monthly_hours	tenure	work_accident	left	\
396	139	3	0	1	
866	128	3	0	1	
1317	127	3	0	1	
1368	132	3	0	1	
1461	142	3	0	1	

	promotion_last_5years	department	salary
396	0	sales	low
866	0	accounting	low
1317	0	sales	medium
1368	0	RandD	low
1461	0	sales	low

The above output shows the first five occurrences of rows that are duplicated farther down in the dataframe. The question is how likely is it that the same employees self-reported the exact same response for every column? And what is the likelihood that these are legitimate entries?

One can perform a likelihood analysis by essentially applying Bayes' theorem and multiplying the probabilities of finding each value in each column, but it does not seem necessary. With several continuous variables across 10 columns, it seems very unlikely that these observations are legitimate. Therefore the entries can be dropped.


```
[10]: # Drop duplicates and save resulting dataframe in a new variable as needed
df = df.drop_duplicates()

# Display first few rows of new dataframe as needed
df.head()
```

```
[10]:
```

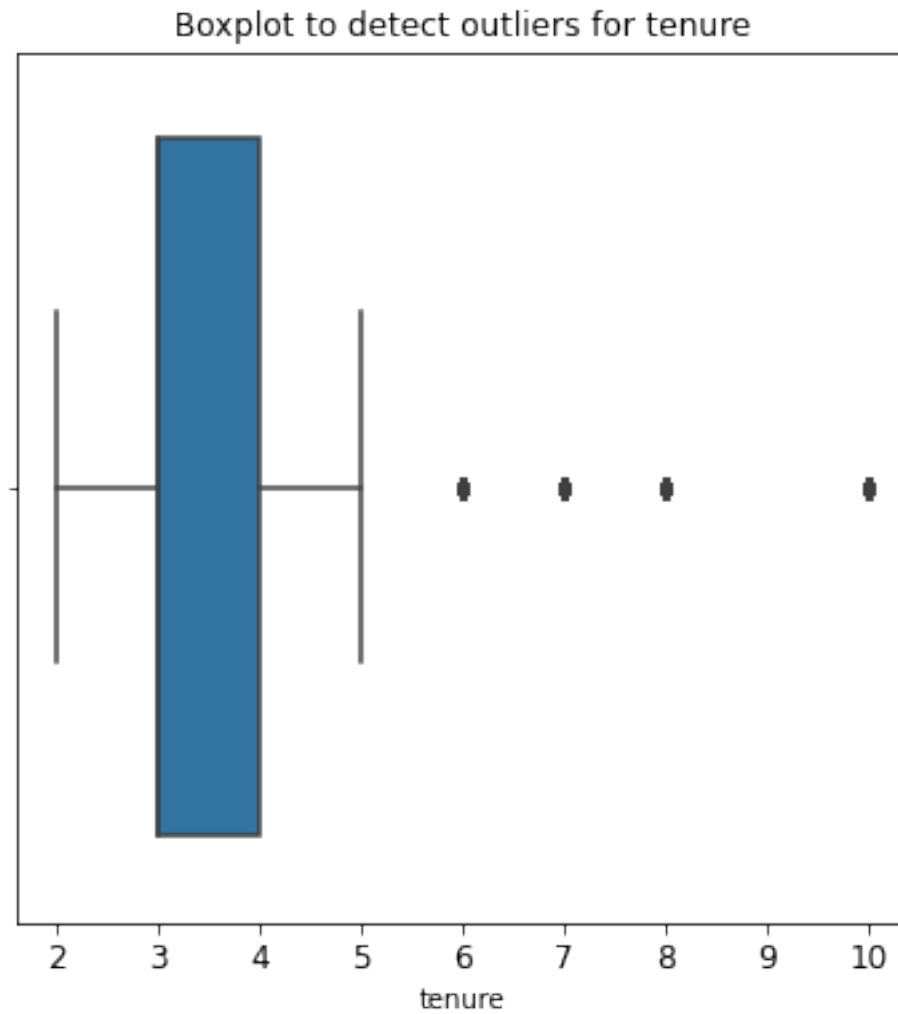
	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

	tenure	work_accident	left	promotion_last_5years	department	salary
0	3	0	1	0	sales	low
1	6	0	1	0	sales	medium
2	4	0	1	0	sales	medium
3	5	0	1	0	sales	low
4	3	0	1	0	sales	low

2.3.6 Check outliers

Check for outliers in the data.

```
[11]: # Create a boxplot to visualize distribution of `tenure` and detect any outliers
plt.figure(figsize=(6,6))
plt.title('Boxplot to detect outliers for tenure', fontsize=12)
plt.xticks(fontsize=12)
plt.yticks(fontsize=12)
sns.boxplot(x=df['tenure'])
plt.show()
```



```
[12]: # Determine the number of rows containing outliers

count_cols = ['tenure']

for column in count_cols:
    q1 = df[column].quantile(0.25)
    q3 = df[column].quantile(0.75)
    iqr = q3 - q1

    # Define the upper limit and lower limit for non-outlier values in `tenure`
    upper_limit = q3 + 1.5 * iqr
    lower_limit = q1 - 1.5 * iqr
    print("Lower limit:", lower_limit)
    print("Upper limit:", upper_limit)
```

```
# Identify subset of data containing outliers in `tenure`
outliers = df[(df[column] > upper_limit) | (df[column] < lower_limit)]

# Count the number of values that exceed the outlier threshold
print(f'Number of rows in the data containing outliers in, {column}:',
      len(outliers))
```

Lower limit: 1.5

Upper limit: 5.5

Number of rows in the data containing outliers in, tenure: 824

Certain types of models are more sensitive to outliers than others. When you get to the stage of building your model, consider whether to remove outliers, based on the type of model you decide to use.

3 pAce: Analyze Stage

- Perform EDA (analyze relationships between variables)

3.1 Step 2. Data Exploration (Continue EDA)

Begin by understanding how many employees left and what percentage of all employees this figure represents.

```
[13]: # Get numbers of people who left vs. stayed
print(df['left'].value_counts())
print()

# Get percentages of people who left vs. stayed
print(df['left'].value_counts(normalize=True))
```

0 10000

1 1991

Name: left, dtype: int64

0 0.833959

1 0.166041

Name: left, dtype: float64

3.1.1 Data visualizations

Now, examine variables that you're interested in, and create plots to visualize relationships between variables in the data.

By creating a stacked boxplot showing `average_monthly_hours` distribution for `number_project`, I can compare the distributions of employees who stayed versus those who left.

Box plots are very useful in visualizing distributions within data, but they can be deceiving without the context of how big the sample size that they represent are. So, making use of a stack histogram to visualize the distribution of `number_project` for those who stayed and those who left will be useful.

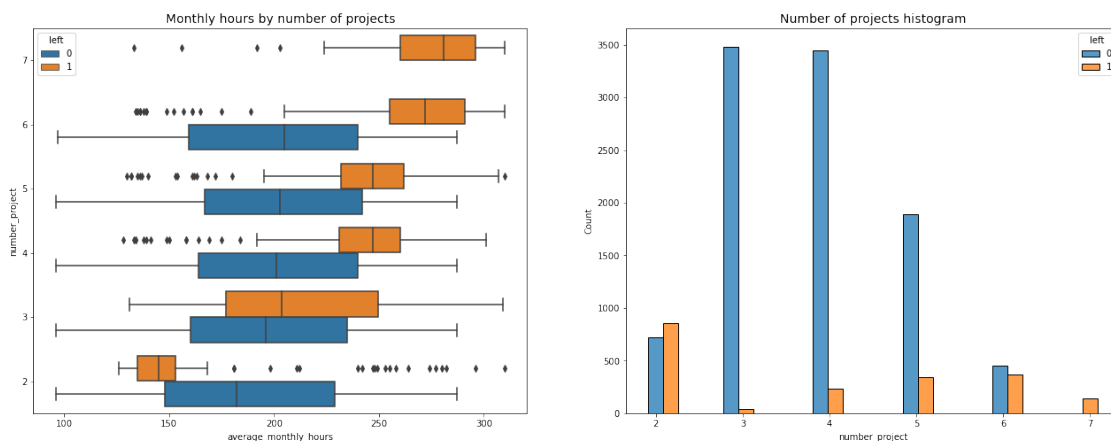
```
[14]: # Create a plot as needed

# Set figure and axes
fig, ax = plt.subplots(1, 2, figsize = (22,8))

# Create boxplot showing `average_monthly_hours` distribution for
↳ `number_project`, comparing employees who stayed versus those who left
sns.boxplot(data=df, x='average_monthly_hours', y='number_project', hue='left',
↳ orient="h", ax=ax[0])
ax[0].invert_yaxis()
ax[0].set_title("Monthly hours by number of projects", fontsize='14')

# Create histogram showing distribution of 'number_project', comparing employees
↳ who stayed versus those who left
tenure_stay = df[df['left']==0]['number_project']
tenure_left = df[df['left']==1]['number_project']
sns.histplot(data=df, x='number_project', hue='left', multiple='dodge',
↳ shrink=2, ax=ax[1])
ax[1].set_title('Number of projects histogram', fontsize=14)

# Display plots
plt.show()
```



It might be natural that people who work on more projects would also work longer hours. This appears to be the case here, with the mean hours of each group (stayed and left) increasing with the number of projects worked. However, a few things stand out from this plot.

1. There are two groups of employees who left the company: (a) those who worked considerably

less than their peers with the same number of projects, and (B) those who worked much more. Of those in group A, it's possible that they were fired. It's also possible that they were fired. It's also possible that this group includes employees who had already given their notice and were assigned fewer hours because they were already on their way out. For those in group B, it's reasonable to infer that they probably quit. The people in group B likely contributed a lot to the projects they worked in; they might have been the largest contributors to their projects.

2. Everyone with seven projects left the company, and the interquartile range of this group and those who left with six projects was ~255-295 hours/month-much more than any other group.
3. The optimal number of projects for employees to work on seems to be 3-4. The ratio of left/stayed is very small for these cohorts.
4. If you assume a work week of 40 hours and two weeks of vacation per year, then the average number of working hours per month of employees working Monday-Friday = $50 \text{ weeks} * 40 \text{ hours per week} / 12 \text{ months} = 166.67 \text{ hours per month}$. This means that, aside from the employees who worked on two projects, every group-even those who didn't leave the company worked considerably more hours than this. It seems that employees here are overworked.

Next, one can confirm that all employees with seven projects left.

```
[15]: # Get value counts of stayed/left for employees with 7 projects
df[df['number_project']==7]['left'].value_counts()
```

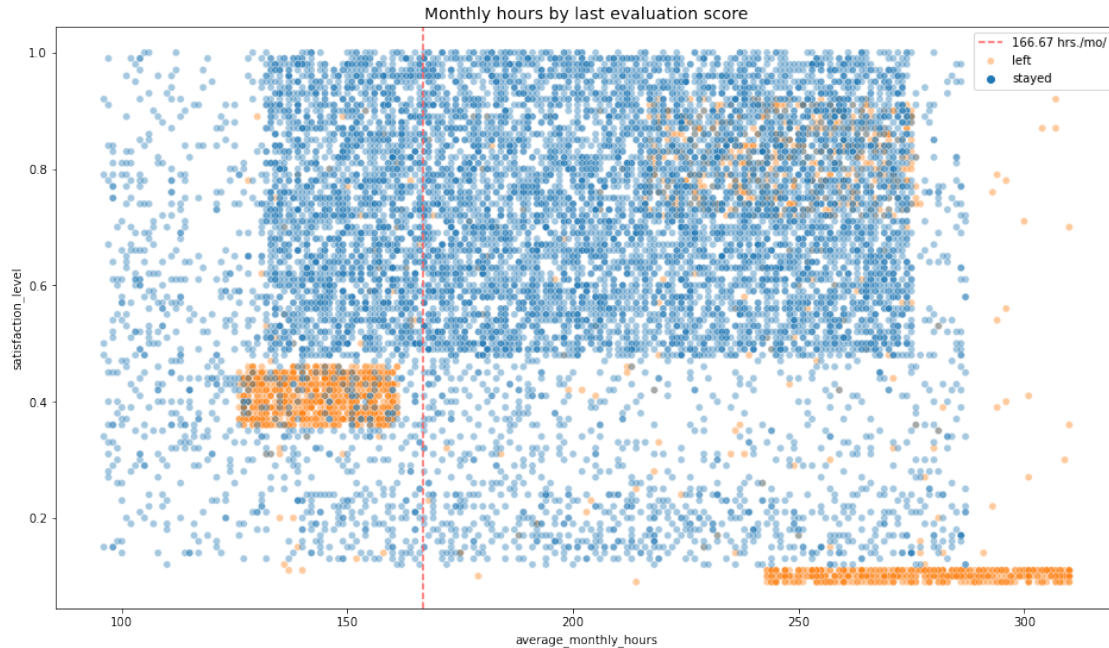
```
[15]: 1      145
      Name: left, dtype: int64
```

This confirms that all employees with 7 projects did leave/

Next, I will examine the average monthly hours versus the satisfaction levels.

```
[16]: # Create a plot as needed
# Create scatterplot of 'average_monthly_hours' versus 'satisfaction_level',
#   ↳comparing employees who stayed versus those who left
plt.figure(figsize=(16, 9))
sns.scatterplot(data=df, x='average_monthly_hours', y='satisfaction_level',
#   ↳hue='left', alpha=0.4)
plt.axvline(x=166.67, color='#ff6361', label='166.67 hrs./mo.', ls='--')
plt.legend(labels=['166.67 hrs./mo/', 'left', 'stayed'])
plt.title('Monthly hours by last evaluation score', fontsize='14')
```

```
[16]: Text(0.5, 1.0, 'Monthly hours by last evaluation score')
```



This scatterplot above shows that there was a sizable group of employees who worked ~240-315 hours per month. 315 hours per month is over 75 hours per week for a whole year. It's likely this is related to their satisfaction levels being close to zero.

The plot also shows groups of people who left, those who had more normal working hours. Even so, their satisfaction was only around 0.4. It's difficult to speculate about why they might have left. It's possible they felt pressure to work more, considering so many of their peers worked more. And that pressure could have lowered their satisfaction levels.

Finally, there is a group who worked ~210-280 hours per month, and they had satisfaction levels ranging ~0.7-0.9.

Note the strange shape of the distribution here. This is indicative of data manipulation or synthetic data.

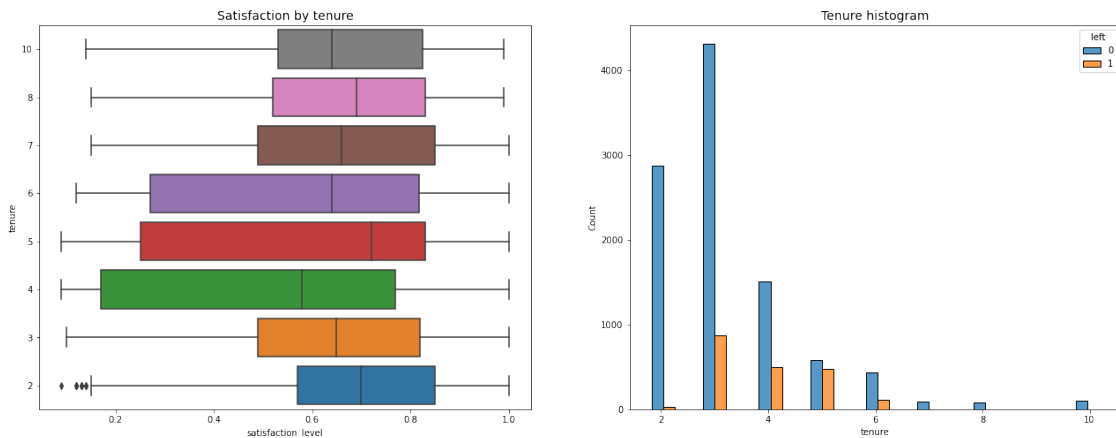
For the next visualization, it might be interesting to visualize satisfaction levels by tenure.

```
[17]: # Create a plot as needed
# Set figure and axes
fig, ax = plt.subplots(1, 2, figsize = (22,8))

# Create boxplot showing distribution of `satisfaction_level` by tenure,
→ comparing employees who stayed versus those who left
sns.boxplot(data=df, x='satisfaction_level', y='tenure', orient="h", ax=ax[0])
ax[0].invert_yaxis()
ax[0].set_title('Satisfaction by tenure', fontsize='14')
```

```
# Create histogram showing distribution of `tenure`, comparing employees who
↳ stayed versus those who left
tenure_stay = df[df['left']==0]['tenure']
tenure_left = df[df['left']==1]['tenure']
sns.histplot(data=df,x='tenure', hue='left', multiple='dodge', shrink=5,
↳ ax=ax[1])
ax[1].set_title('Tenure histogram', fontsize='14')

plt.show();
```



There are a few observations from this plot.

- Employees who left fall into two general categories: dissatisfied employees with shorter tenures and very satisfied employees with medium-length tenures.
- Four-year employees who left seem to have an unusually low satisfaction level. It's worth investigating changes to company policy that might have affected people specifically at the four-year mark, if possible.
- The longest-tenured employees didn't leave. Their satisfaction level aligned with those of newer employees who stayed.
- The histogram shows that there are relatively few longer-tenured employees. It's possible that they're the highest-ranking, higher-paid employees.

For the next step in analyzing the data, one could calculate the mean and the median satisfaction scores of employees who left and those who didn't.

```
[18]: # Calculate mean and median satisfaction scores of employees who left and those
↳ who stayed
df.groupby(['left'])['satisfaction_level'].agg([np.mean,np.median])
```

```
[18]:
```

	mean	median
left		
0	0.667365	0.69
1	0.440271	0.41

As expected, the mean and median satisfaction scores of employees who left are lower than those of employees who stayed. Interestingly, among employees who stayed, the mean satisfaction score appears to be slightly below the median score. This indicates that satisfaction levels among those who stayed might be skewed to the left.

Next, you could examine salary levels for different tenures.

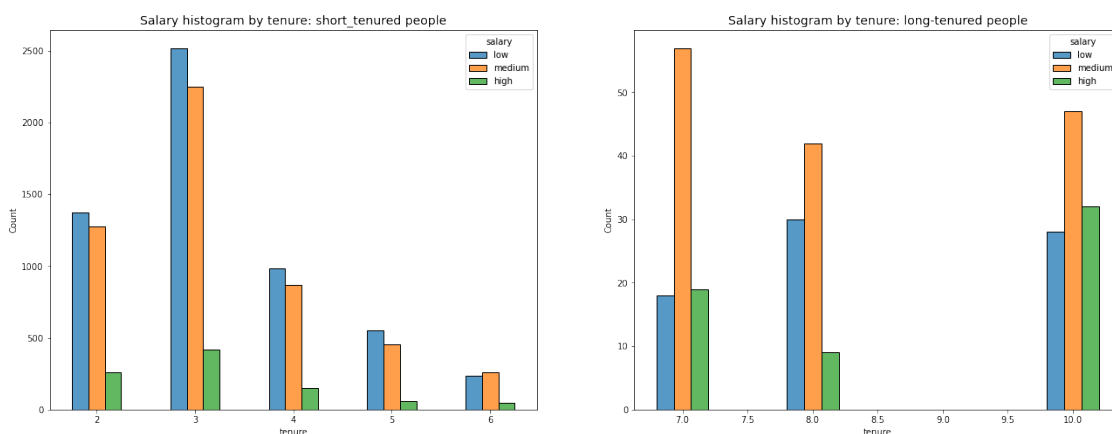
```
[19]: # Create a plot as needed
# Set figure size
fig, ax = plt.subplots(1, 2, figsize = (22,8))

# Define short-tenured employees
tenure_short = df[df['tenure'] < 7]

# Define long-tenured employees
tenure_long = df[df['tenure'] > 6]

# Plot short-tenured histogram
sns.histplot(data=tenure_short, x='tenure', hue='salary', discrete=1,
             hue_order=['low', 'medium', 'high'], multiple='dodge', shrink=.5,
             →ax=ax[0])
ax[0].set_title('Salary histogram by tenure: short_tenured people',
               →fontsize='14')

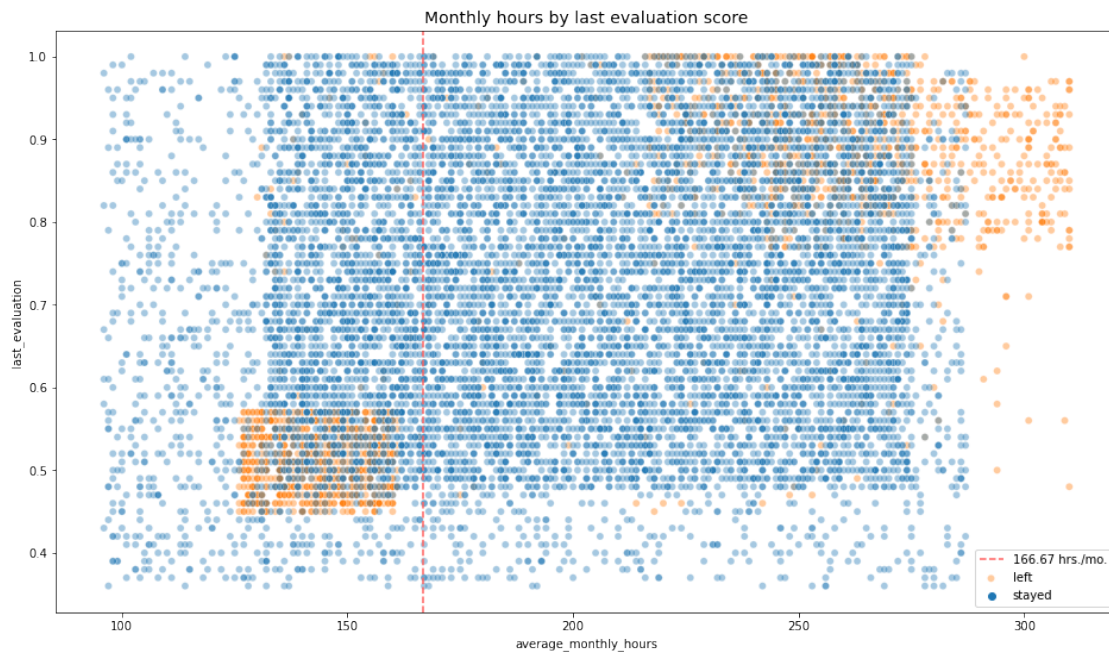
# Plot long-tenured histogram
sns.histplot(data=tenure_long, x='tenure', hue='salary', discrete=1,
             hue_order=['low', 'medium', 'high'], multiple='dodge', shrink=.4,
             →ax=ax[1])
ax[1].set_title('Salary histogram by tenure: long-tenured people',
               →fontsize='14');
```



The plots above show that long-tenured employees were not disproportionately composed of higher_paid employees.

Next, one could explore whether there's a correlation between working long hours and receiving high evaluation scores. You could create a scatterplot of `average_monthly_hours` versus `last_evaluation`.

```
[20]: # Create a plot as needed
# Create scatterplot of `average_monthly_hours` versus `last_evaluation`
plt.figure(figsize=(16, 9))
sns.scatterplot(data=df, x='average_monthly_hours', y='last_evaluation',
               hue='left', alpha=0.4)
plt.axvline(x=166.67, color='#ff6361', label='166.67 hrs./mo.', ls='--')
plt.legend(labels=['166.67 hrs./mo.', 'left', 'stayed'])
plt.title('Monthly hours by last evaluation score', fontsize='14');
```

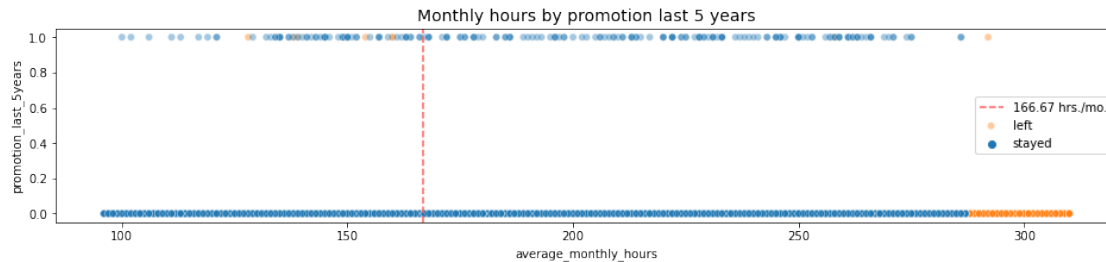


The following observations can be made from the scatterplot above:

- The scatterplot indicates two groups of employees who left: overworked employees who performed very well and employees who worked slightly under the normal monthly average of 166.67 hours with lower evaluation scores.
- There seems to be a correlation between hours worked and evaluations score.
- There isn't a high percentage of employees in the upper left quadrant of this plot: but working long hours doesn't guarantee a good evaluation score.
- Most of the employees in this company work well over 167 hours per month.

Next, one could examine whether employees who worked very long hours were promoted in the last five years.

```
[21]: # Create a plot as needed
# Create plot to examine relationship between `average_monthly_hours` and
↳ `promotion_last_5years`
plt.figure(figsize=(16, 3))
sns.scatterplot(data=df, x='average_monthly_hours', y='promotion_last_5years',
↳ hue='left', alpha=0.4)
plt.axvline(x=166.67, color='#ff6361', ls='--')
plt.legend(labels=['166.67 hrs./mo.', 'left', 'stayed'])
plt.title('Monthly hours by promotion last 5 years', fontsize='14');
```



The plot above shows the following:

- very few employees who were promoted in the last five years left
- very few employees who worked the most hours were promoted
- all of the employees who left were working the longest hours

next, one could inspect how the employees who left are distributed across departments.

```
[22]: # Display counts for each department
df['department'].value_counts()
```

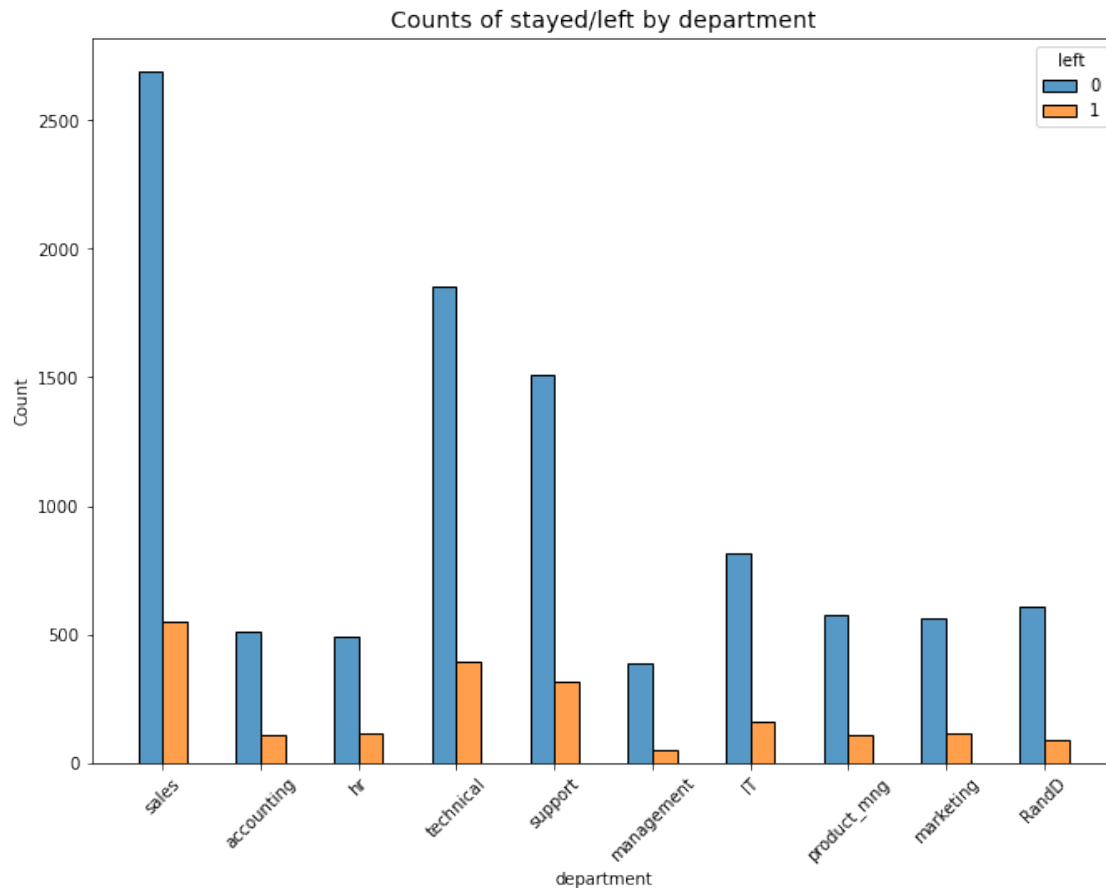
```
[22]: sales          3239
      technical      2244
      support        1821
      IT             976
      RandD          694
      product_mng    686
      marketing      673
      accounting     621
      hr             601
      management     436
      Name: department, dtype: int64
```

```
[23]: # Create a plot as needed
# Create stacked histogram to compare department distribution of employees who
↳ left to that of employees who didn't
plt.figure(figsize=(11,8))
sns.histplot(data=df, x='department', hue='left', discrete=1,
```

```

hue_order=[0, 1], multiple='dodge', shrink=.5)
plt.xticks(rotation='45')
plt.title('Counts of stayed/left by department', fontsize=14);

```



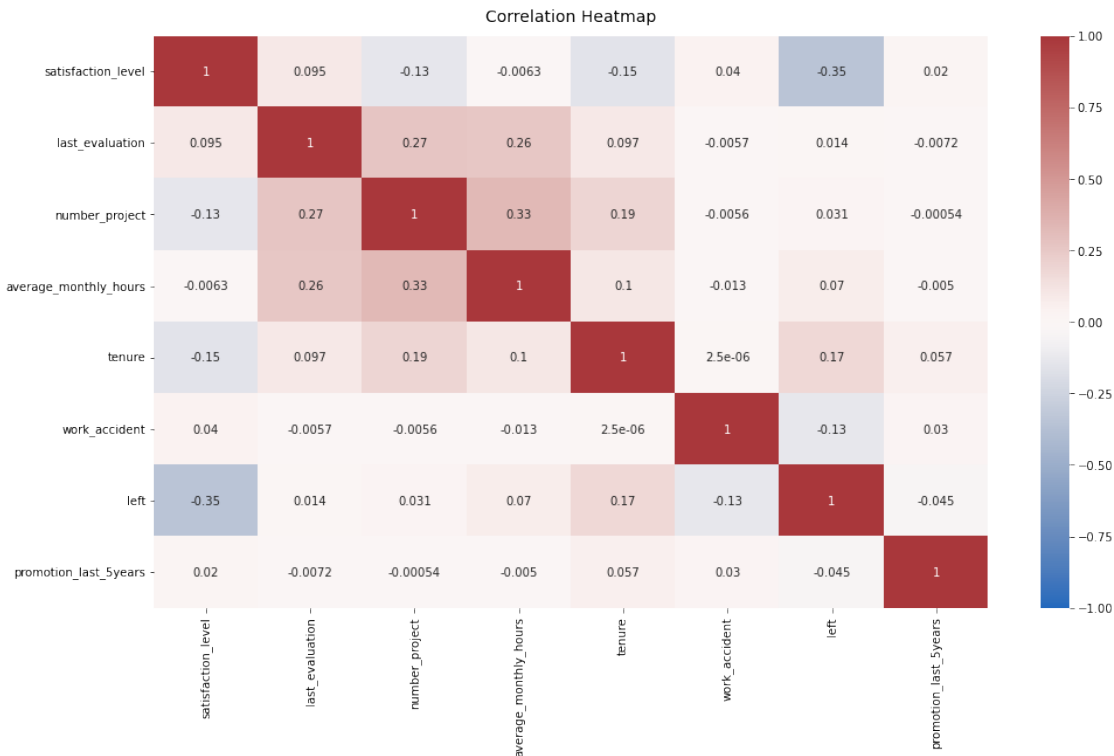
There doesn't seem to be any department that differs significantly in its proportion of employees who left to those who stayed.

lastly, I will check for strong correlation between variables in the data.

```

[24]: # Plot a correlation heatmap
plt.figure(figsize=(16, 9))
heatmap = sns.heatmap(df.corr(), vmin=-1, vmax=1, annot=True, cmap=sns.
    color_palette("vlag", as_cmap=True))
heatmap.set_title('Correlation Heatmap', fontdict={'fontsize':14}, pad=12);

```



The correlation heatmap confirms that the number of projects, monthly hours, and evaluation scores all have positive correlation with each other, and whether an employee leaves is negatively correlated with their satisfaction level.

3.1.2 Insights

It appears that employees are leaving as a result of poor management. Leaving is tied to longer working hours, many projects, and generally lower satisfaction levels. It can be ungratifying to work longer hours and not receive promotions or good evaluation scores. There's a sizable group of employees at this company who are probably burned out. It also appears that if an employee has spent more than six years at the company, they tend not to leave.

4 paCe: Construct Stage

- Determine which models are most appropriate
- Construct the model
- Confirm model assumptions
- Evaluate model results to determine how well your model fits the data

Recall model assumptions

Logistic Regression model assumptions - Outcome variable is categorical - Observations are

independent of each other - No severe multicollinearity among X variables - No extreme outliers
- Linear relationship between each X variable and the logit of the outcome variable - Sufficiently large sample size

Reflect on these questions as you complete the constructing stage.

- Do you notice anything odd?
- Which independent variables did you choose for the model and why?
- Are each of the assumptions met?
- How well does your model fit the data?
- Can you improve it? Is there anything you would change about the model?
- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

[Double-click to enter your responses here.]

4.1 Step 3. Model Building, Step 4. Results and Evaluation

- Fit a model that predicts the outcome variable using two or more independent variables
- Check model assumptions
- Evaluate the model

4.1.1 Identify the type of prediction task.

The goal is to predict whether an employee leaves the company, which is a categorical outcome variable. So this task involves classification. More specifically, this involves binary classification, since the outcome variable `left` can be either 1 (indicating employee left) or 0 (indicating employee didn't leave).

4.1.2 Identify the types of models most appropriate for this task.

[Double-click to enter your responses here.]

4.1.3 Modeling

Since the variables I want to predict (whether an employee leaves the company) is categorical, I can either build a Logistic Regression model, or a Tree-based Machine Learning model.

For this project I have decided to implement both and compare their results.

4.1.4 Modeling Approach A: Logistic Regression Model

Logistic regression

Note that binomial logistic regression suits the task because it involves binary classification.

Before splitting the data, encode the non-numeric variables. There are two: `department` and `salary`.

`department` is a categorical variable, which means you can dummy it for modeling.

`salary` is categorical too, but it's ordinal. There's a hierarchy to the categoric, so it's better not to dummy this column, but rather to convert the levels to numbers, 0-2.

```
[25]: # Copy the dataframe
df_enc = df.copy()

# Encode the `salary` column as an ordinal numeric category
df_enc['salary'] = (
    df_enc['salary'].astype('category')
    .cat.set_categories(['low', 'medium', 'high'])
    .cat.codes
)

# Dummy encode the `department` column

df_enc = pd.get_dummies(df_enc, drop_first=False)

# Display the new dataframe
df_enc.head()
```

```
[25]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
1	0.80	0.86	5	262	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	

	tenure	work_accident	left	promotion_last_5years	salary	department_IT	\
0	3	0	1	0	0	0	
1	6	0	1	0	1	0	
2	4	0	1	0	1	0	
3	5	0	1	0	0	0	
4	3	0	1	0	0	0	

	department_RandD	department_accounting	department_hr	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

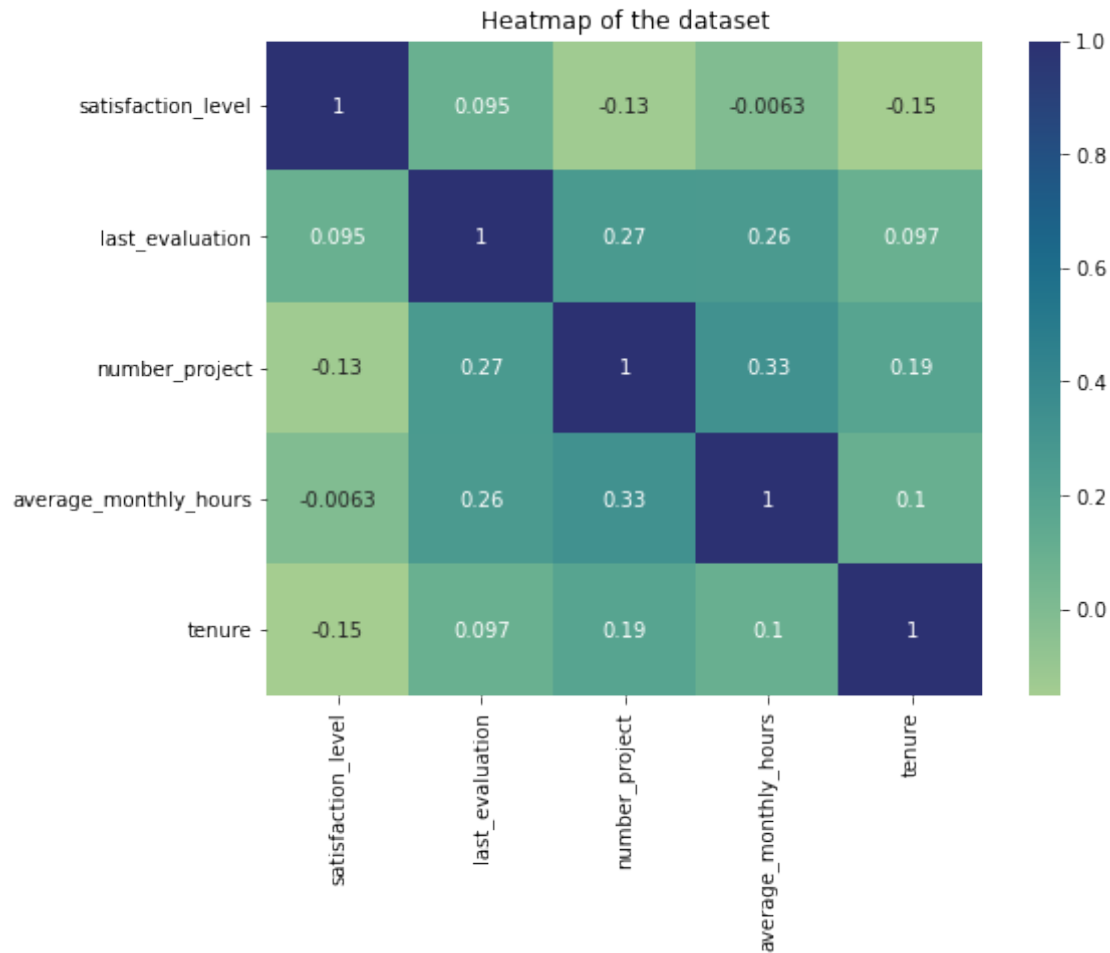
	department_management	department_marketing	department_product_mng	\
0	0	0	0	

1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

	department_sales	department_support	department_technical
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0

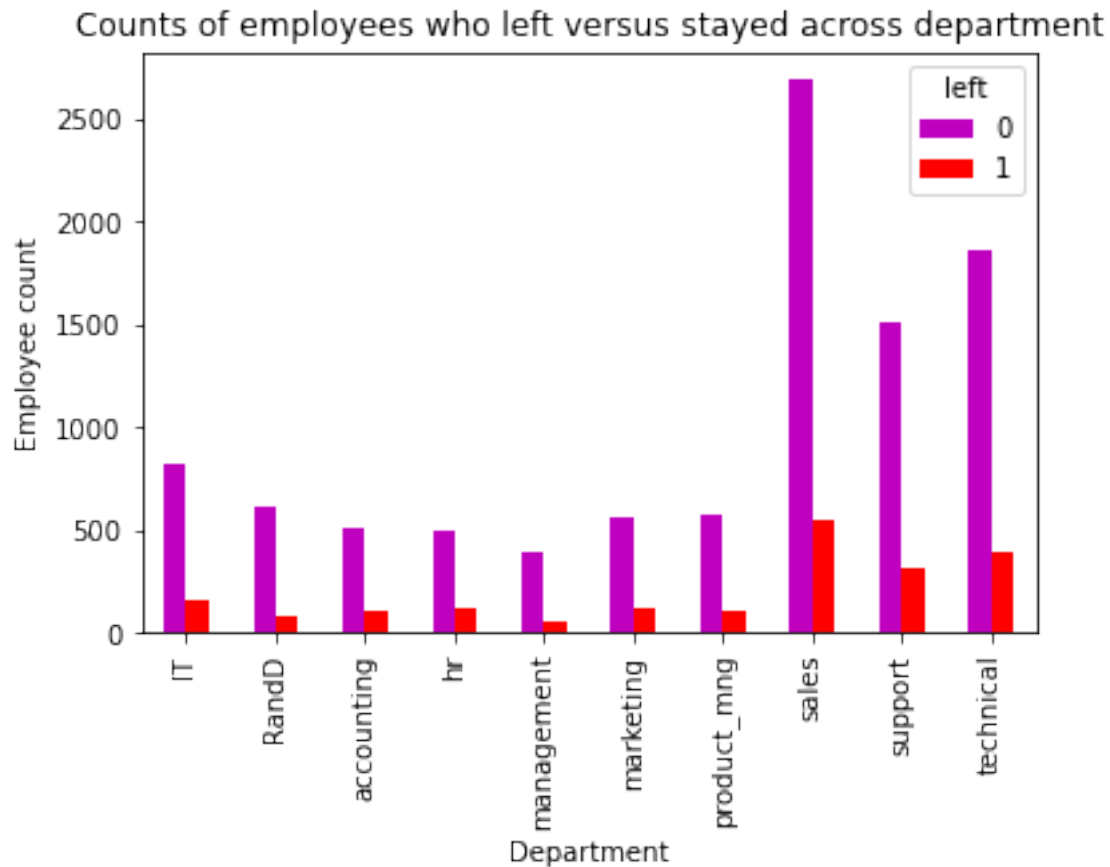
Create a heatmap to visualize how correlated variables are.

```
[26]: # Create a heatmap to visualize how correlated variables are
plt.figure(figsize=(8, 6))
sns.heatmap(df_enc[['satisfaction_level', 'last_evaluation', 'number_project',
    ↪ 'average_monthly_hours', 'tenure']]
            .corr(), annot=True, cmap="crest")
plt.title("Heatmap of the dataset")
plt.show()
```



Next, I will create a stacked bar plot to visualize the number of employees across departments, comparing those who left with those who didn't.

```
[27]: # Create a stacked bar plot to visualize number of employees across department,
      ↪ comparing those who left with those who didn't
      # In the legend, 0 (purple color) represents employees who did not leave, 1
      ↪ (red color) represents employees who left
      pd.crosstab(df['department'], df['left']).plot(kind='bar', color='mr')
      plt.title('Counts of employees who left versus stayed across department')
      plt.ylabel('Employee count')
      plt.xlabel('Department')
      plt.show()
```

Since logistic regression is quite sensitive to outliers, it would be a good idea at this stage to remove the outliers in the `tenure` column that were identified earlier.

```
[28]: # Select rows without outliers in `tenure` and save resulting dataframe in a
      ↪ new variable
df_logreg = df_enc[(df_enc['tenure'] >= lower_limit) & (df_enc['tenure'] <=
      ↪ upper_limit)]

# Display first few rows of new dataframe
df_logreg.head()
```

```
[28]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	
5	0.41	0.50	2	153	

	tenure	work_accident	left	promotion_last_5years	salary	department_IT	\
0	3	0	1	0	0	0	

2	4	0	1	0	1	0
3	5	0	1	0	0	0
4	3	0	1	0	0	0
5	3	0	1	0	0	0

	department_RandD	department_accounting	department_hr	\
0	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	0	0	0	

	department_management	department_marketing	department_product_mng	\
0	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	0	0	0	

	department_sales	department_support	department_technical
0	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0
5	1	0	0

Isolate the outcome variable, which is the variable the model should predict.

```
[29]: # Isolate the outcome variable
y = df_logreg['left']

# Display first few rows of the outcome variable
y.head()
```

```
[29]: 0    1
      2    1
      3    1
      4    1
      5    1
      Name: left, dtype: int64
```

Select the feature to use in the model. Consider which variables will help predict the outcome variables, left.

```
[30]: # Select the features to use in the model
X = df_logreg.drop('left', axis=1)

# Display the first few rows of the selected features
```

```
X.head()
```

```
[30]:
```

	satisfaction_level	last_evaluation	number_project	average_monthly_hours	\
0	0.38	0.53	2	157	
2	0.11	0.88	7	272	
3	0.72	0.87	5	223	
4	0.37	0.52	2	159	
5	0.41	0.50	2	153	

	tenure	work_accident	promotion_last_5years	salary	department_IT	\
0	3	0	0	0	0	
2	4	0	0	1	0	
3	5	0	0	0	0	
4	3	0	0	0	0	
5	3	0	0	0	0	

	department_RandD	department_accounting	department_hr	\
0	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	0	0	0	

	department_management	department_marketing	department_product_mng	\
0	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	
5	0	0	0	

	department_sales	department_support	department_technical
0	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0
5	1	0	0

Next, I will split the data into a training and testing set. Since the classes are unbalanced, I will stratify based on the values of y.

```
[31]: # Split the data into training set and testing set
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↪stratify=y, random_state=42)
```

Construct a logistic regression model and fit it to the training dataset.

```
[32]: # Construct a logistic regression model and fit it to the training dataset
log_clf = LogisticRegression(random_state=42, max_iter=500).fit(X_train,
→y_train)
```

Test the logistic regression model: use the model to make predictions on the test set.

```
[33]: # Use the logistic regression model to get predictions on the test set
y_pred = log_clf.predict(X_test)
```

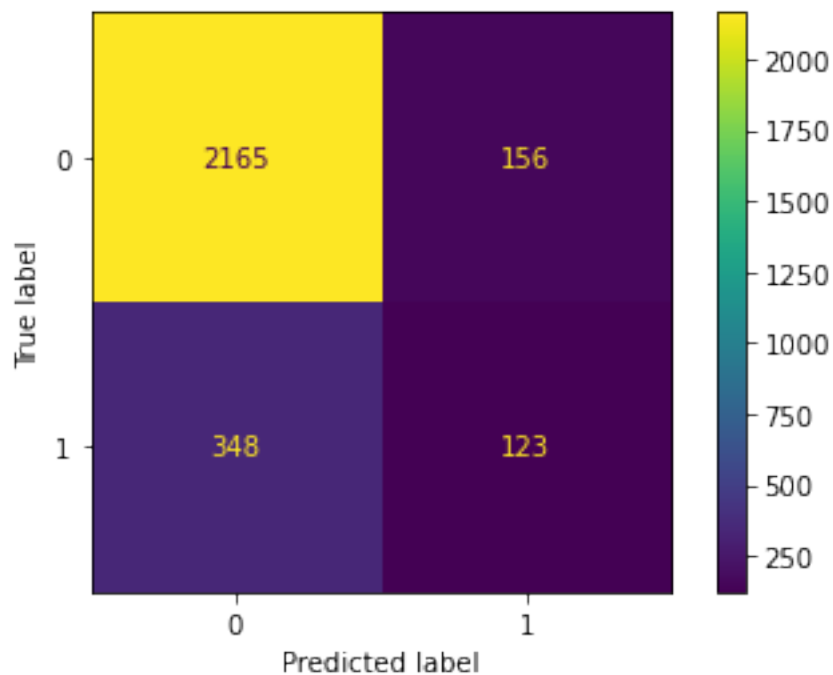
Create a confusion matrix to visualize the results of the logistic regression model.

```
[34]: # Compute values for confusion matrix
log_cm = confusion_matrix(y_test, y_pred, labels=log_clf.classes_)

# Create display of confusion matrix
log_disp = ConfusionMatrixDisplay(confusion_matrix=log_cm,
                                  display_labels=log_clf.classes_)

# Plot confusion matrix
log_disp.plot(values_format='')

# Display plot
plt.show()
```



The upper-left quadrant displays the number of true negatives. The upper-right quadrant displays the number of false positives. The bottom-left quadrant displays the number of false negatives. The bottom-right quadrant displays the number of true positives.

True negatives: The number of people who did not leave that the model accurately predicted did not leave.

False positives: The number of people who did not leave the model inaccurately predicted did not leave.

False negatives: The number of people who left that the model inaccurately predicted as leaving.

True positive: The number of people how left the model accurately predicted as leaving.

A perfect model would yield all true negatives and true positives, and no false negatives or false positives.

Create a classification report that includes precision, recall, f1-score, and accuracy metrics to evaluate the performance of the logistic regression model.

Check the class balance in the data. In other words, check the value counts in the `left` column. Since this is a binary classification task, the class balance informs the way you interpret accuracy metrics.

```
[35]: df_logreg['left'].value_counts(normalize=True)
```

```
[35]: 0    0.831468
      1    0.168532
      Name: left, dtype: float64
```

There is an approximately 83%-17% split. So the data is not perfectly balanced, but it is not too imbalanced. If it was more severely imbalanced, you might want to resample the data to make it more balanced. In this case, you can use this data without modifying the class balance and continue evaluating the model.

```
[36]: # Create classification report for logistic regression model
      target_names = ['Predicted would not leave', 'Predicted would leave']
      print(classification_report(y_test, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
Predicted would not leave	0.86	0.93	0.90	2321
Predicted would leave	0.44	0.26	0.33	471
accuracy			0.82	2792
macro avg	0.65	0.60	0.61	2792
weighted avg	0.79	0.82	0.80	2792

The classification report above shows that the logistic regression model achieved a precision of 79%, recall of 82%, f1_score of 80%(all weighted averages), and accuracy of 82%. However, if it's most important to predict employees who leave, then the scores are significantly lower.

4.1.5 Modeling Approach B: Tree-based Model

This approach covers implementation of Decision Tree and Random Forest.

Isolate the outcome variable.

```
[37]: # Isolate the outcome variable
y = df_enc['left']

# Display the first few rows of 'y'
y.head()
```

```
[37]: 0    1
      1    1
      2    1
      3    1
      4    1
      Name: left, dtype: int64
```

Select the features.

```
[38]: # Select the features
X = df_enc.drop('left', axis=1)

# Display the first few rows of 'X'
X.head()
```

```
[38]: satisfaction_level  last_evaluation  number_project  average_monthly_hours  \
0                0.38              0.53                2                157
1                0.80              0.86                5                262
2                0.11              0.88                7                272
3                0.72              0.87                5                223
4                0.37              0.52                2                159
```

```
      tenure  work_accident  promotion_last_5years  salary  department_IT  \
0          3              0                    0        0                0
1          6              0                    0        1                0
2          4              0                    0        1                0
3          5              0                    0        0                0
4          3              0                    0        0                0
```

```
      department_RandD  department_accounting  department_hr  \
0                    0                    0                0
1                    0                    0                0
2                    0                    0                0
3                    0                    0                0
4                    0                    0                0
```

```
      department_management  department_marketing  department_product_mng  \
```

0	0	0	0
1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

	department_sales	department_support	department_technical
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0

Split the data into training, validating, and test sets.

```
[39]: # Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↳stratify=y, random_state=0)
```

Decision tree - Round 1

Construct a decision tree model and set up cross-validated grid-search to exhaustively search for the best model parameters.

```
[40]: # Instantiate model
tree = DecisionTreeClassifier(random_state=0)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [4, 6, 8, None],
             'min_samples_leaf': [2, 5, 1],
             'min_samples_split': [2, 4, 6]
            }

# Assign a dictionary of scoring metrics to capture
scoring = {'accuracy', 'precision', 'recall', 'f1', 'roc_auc'}

# Instantiate GridSearch
tree1 = GridSearchCV(tree, cv_params, scoring=scoring, cv=4, refit='roc_auc')
```

Fit the decision tree model to the training data.

```
[41]: %%time
tree1.fit(X_train, y_train)
```

CPU times: user 2.89 s, sys: 0 ns, total: 2.89 s
Wall time: 2.89 s

```
[41]: GridSearchCV(cv=4, error_score=nan,
                  estimator=DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None,
                                                    criterion='gini', max_depth=None,
```

```

max_features=None,
max_leaf_nodes=None,
min_impurity_decrease=0.0,
min_impurity_split=None,
min_samples_leaf=1,
min_samples_split=2,
min_weight_fraction_leaf=0.0,
presort='deprecated',
random_state=0, splitter='best'),

iid='deprecated', n_jobs=None,
param_grid={'max_depth': [4, 6, 8, None],
            'min_samples_leaf': [2, 5, 1],
            'min_samples_split': [2, 4, 6]},
pre_dispatch='2*n_jobs', refit='roc_auc', return_train_score=False,
scoring={'recall', 'accuracy', 'roc_auc', 'f1', 'precision'},
verbose=0)

```

Identify the optimal values for the decision tree parameters.

```
[42]: # Check best parameters
tree1.best_params_
```

```
[42]: {'max_depth': 4, 'min_samples_leaf': 5, 'min_samples_split': 2}
```

Identify the best AUC score achieved by the decision tree model on the training set.

```
[43]: # Check best AUC score on CV
tree1.best_score_
```

```
[43]: 0.969819392792457
```

This is a strong AUC score, which shows that this model can predict employees who will leave very well.

Next, write a function that will help extract all the scores from the grid search.

```
[44]: def make_results(model_name:str, model_object, metric:str):
    '''
    Arguments:
        model_name (string): what you want the model to be called in the output_
        ↪ table
        model_object: a fit GridSearchCV object
        metric (string): precision, recall, f1, accuracy, or auc

    Returns a pandas df with the F1, recall, precision, accuracy, and auc scores
    for the model with the best mean 'metric' score across all validation folds.
    ↪
    '''
```



```

# Create dictionary that maps input metric to actual metric name in
↳GridSearchCV
metric_dict = {'auc': 'mean_test_roc_auc',
               'precision': 'mean_test_precision',
               'recall': 'mean_test_recall',
               'f1': 'mean_test_f1',
               'accuracy': 'mean_test_accuracy'
              }

# Get all the results from the CV and put them in a df
cv_results = pd.DataFrame(model_object.cv_results_)

# Isolate the row of the df with the max(metric) score
best_estimator_results = cv_results.iloc[cv_results[metric_dict[metric]].
↳idxmax(), :]

# Extract Accuracy, precision, recall, and f1 score from that row
auc = best_estimator_results.mean_test_roc_auc
f1 = best_estimator_results.mean_test_f1
recall = best_estimator_results.mean_test_recall
precision = best_estimator_results.mean_test_precision
accuracy = best_estimator_results.mean_test_accuracy

# Create table of results
table = pd.DataFrame()
table = pd.DataFrame({'model': [model_name],
                     'precision': [precision],
                     'recall': [recall],
                     'F1': [f1],
                     'accuracy': [accuracy],
                     'auc': [auc]
                    })

return table

```

```

[45]: # Get all CV scores
tree1_cv_results = make_results('decision treecv', tree1, 'auc')
tree1_cv_results

```

```

[45]:          model  precision    recall      F1  accuracy      auc
0  decision treecv   0.914552  0.916949  0.915707  0.971978  0.969819

```

All of these scores from the decision tree model are strong indicators of good model performance.

Decision trees can be vulnerable to overfitting, and random forests avoid overfitting by incorporating multiple trees to make predictions. Next, I will construct a random forest model.

Random forest - Round 1 Construct a random forest model and set up cross-validated grid-search to exhaustively search for the best model parameters.

```
[46]: # Instantiate model
rf = RandomForestClassifier(random_state=0)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [3,5, None],
             'max_features': [1.0],
             'max_samples': [0.7, 1.0],
             'min_samples_leaf': [1,2,3],
             'min_samples_split': [2,3,4],
             'n_estimators': [300, 500],
             }

# Assign a dictionary of scoring metrics to capture
scoring = {'accuracy', 'precision', 'recall', 'f1', 'roc_auc'}

# Instantiate GridSearch
rf1 = GridSearchCV(rf, cv_params, scoring=scoring, cv=4, refit='roc_auc')
```

Fit the random forest to the training data

```
[47]: %%time
      rf1.fit(X_train, y_train)
```

CPU times: user 9min 12s, sys: 0 ns, total: 9min 12s

Wall time: 9min 12s

```
[47]: GridSearchCV(cv=4, error_score=nan,
                  estimator=RandomForestClassifier(bootstrap=True, ccp_alpha=0.0,
                                                    class_weight=None,
                                                    criterion='gini', max_depth=None,
                                                    max_features='auto',
                                                    max_leaf_nodes=None,
                                                    max_samples=None,
                                                    min_impurity_decrease=0.0,
                                                    min_impurity_split=None,
                                                    min_samples_leaf=1,
                                                    min_samples_split=2,
                                                    min_weight_fraction_leaf=0.0,
                                                    n_estimators=100, n_jobs=None,...
                                                    verbose=0, warm_start=False),
                  iid='deprecated', n_jobs=None,
                  param_grid={'max_depth': [3, 5, None], 'max_features': [1.0],
                              'max_samples': [0.7, 1.0],
                              'min_samples_leaf': [1, 2, 3],
                              'min_samples_split': [2, 3, 4],
```

```

        'n_estimators': [300, 500]],
    pre_dispatch='2*n_jobs', refit='roc_auc', return_train_score=False,
    scoring={'recall', 'accuracy', 'roc_auc', 'f1', 'precision'},
    verbose=0)

```

```

[48]: # Define a path to the folder where model should be saved
path = '/home/jovyan/work/'

```

Define functions to pickle the model and read in the model.

```

[49]: def write_pickle(path, model_object, save_as: str):
    '''
    In:
        path:          path of folder where pickle is saved
        model_object:  a model to pickle
        save_as:       filename for the saved model

    Out: A call to pickle the model in the folder indicated
    '''

    with open(path + save_as + '.pickle', 'wb') as to_write:
        pickle.dump(model_object, to_write)

```

```

[50]: def read_pickle(path, saved_model_name:str):
    '''
    In:
        path:          path to folder where you want to read from
        saved_model_name: filename of pickled model you want to read in

    Out:
        model: the pickled model
    '''

    with open(path + saved_model_name + '.pickle', 'rb') as to_read:
        model = pickle.load(to_read)

    return model

```

Use the functions defined above to save the model in a pickle file and then read it.

```

[51]: # Write pickle
write_pickle(path, rf1, 'hr_rf1')

```

```

[52]: # Read pickle
rf1= read_pickle(path, 'hr_rf1')

```

Identify the best AUC score achieved by the random forest model on the training set.

```
[53]: # Check best AUC score on CV
      rf1.best_score_
```

```
[53]: 0.9804250949807172
```

Identify the optimal values for the parameters of the random forest model.

```
[54]: # Check the best params
      rf1.best_params_
```

```
[54]: {'max_depth': 5,
      'max_features': 1.0,
      'max_samples': 0.7,
      'min_samples_leaf': 1,
      'min_samples_split': 4,
      'n_estimators': 500}
```

Collect the evaluation scores on the training set for the decision tree and random forest models.

```
[55]: # Get all CV scores
      rf1_cv_results = make_results('random forest cv', rf1, 'auc')
      print(tree1_cv_results)
      print(rf1_cv_results)
```

	model	precision	recall	F1	accuracy	auc
0	decision treecv	0.914552	0.916949	0.915707	0.971978	0.969819
	model	precision	recall	F1	accuracy	auc
0	random forest cv	0.950023	0.915614	0.932467	0.977983	0.980425

The evaluation scores of the random forest model are better than those of the decision tree model, with the exception of recall(the recall score of the. random forest model is approximately 0.001 lower, which is a negligible amount). This indicates that the random forest model mostly outperformed the decision tree model.

Next, I will evaluate the final model of the test set.

Define a function that gets all scores from a model's predictions.

```
[56]: def get_scores(model_name:str, model, X_test_data, y_test_data):
      """
      Generate a table of test scores.

      In:
          model_name (string): How you want your model to be named in the output_
      ↪table
          model: A fit GridSearchCV object
          X_test_data: numpy array of X_test data
          y_test_data: numpy array of y_test data
```

```

Out: pandas df of precision, recall, f1, accuracy, and AUC scores for your_
↪model
'''

preds = model.best_estimator_.predict(X_test_data)

auc = roc_auc_score(y_test_data, preds)
accuracy = accuracy_score(y_test_data, preds)
precision = precision_score(y_test_data, preds)
recall = recall_score(y_test_data, preds)
f1 = f1_score(y_test_data, preds)

table = pd.DataFrame({'model': [model_name],
                        'precision': [precision],
                        'recall': [recall],
                        'f1': [f1],
                        'accuracy': [accuracy],
                        'AUC': [auc]
                      })

return table

```

Now use the best performance model to predict on the test set.

```

[57]: rf1_test_scores = get_scores('random forest1 test', rf1, X_test, y_test)
      rf1_test_scores

```

```

[57]:
      model  precision  recall    f1  accuracy    AUC
0  random forest1 test    0.964211  0.919679  0.941418  0.980987  0.956439

```

The test scores are very similar to the validation scores, which is good. This appears to be a strong model. Since this test set was only used for this model, one can be more confident that the model's performance on this data is representative of how it will perform on new, unseen data.

Feature Engineering

Due to the high evaluation scores. There is a chance that there is some data leakage occurring. Data leakage is when you use data to train your model that should not be used during training, either because it appears in the test data or because it's not data that you'd expect to have when the model is actually deployed. Training a model with leaked data can give an unrealistic score that is not replicated in production.

In this case, it's likely that the company won't have satisfaction levels reported for all of its employees. It's also possible that the `average_monthly_hours` column is a source of some data leakage. If employees have already decided upon quieting, or have already been identified by management as people to be fired, they may be working fewer hours.

The first round of decision tree and random forest models included all variables as features. This next round will incorporate feature engineering to build improved models.

One could proceed by dropping `satisfaction_level` and create a new feature that roughly captures whether an employee is overworked. This new feature can be called `overworked`. It will be a binary variable.

```
[58]: # Drop `satisfaction_level` and save resulting dataframe into a new variable
df2 = df_enc.drop('satisfaction_level', axis=1)

# Display first few rows of new dataset
df2.head()
```

```
[58]:
```

	last_evaluation	number_project	average_monthly_hours	tenure	\
0	0.53	2	157	3	
1	0.86	5	262	6	
2	0.88	7	272	4	
3	0.87	5	223	5	
4	0.52	2	159	3	

	work_accident	left	promotion_last_5years	salary	department_IT	\
0	0	1	0	0	0	
1	0	1	0	1	0	
2	0	1	0	1	0	
3	0	1	0	0	0	
4	0	1	0	0	0	

	department_RandD	department_accounting	department_hr	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	department_management	department_marketing	department_product_mng	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	department_sales	department_support	department_technical
0	1	0	0
1	1	0	0
2	1	0	0
3	1	0	0
4	1	0	0

```
[59]: # Create `overworked` column. For now, it's identical to avrage monthly hours.
df2['overworked'] = df2['average_monthly_hours']
```

```
# Inspect max and min average monthly hours values
print('Max hours:', df2['overworked'].max())
print('Min hours:', df2['overworked'].min())
```

Max hours: 310

Min hours: 96

166.67 is approximately the average number of monthly hours for someone who works 50 weeks per year, 5 days per week, 8 hours per day.

One could define being overworked as working more than 175 hours per month on average.

To make the `overworked` column binary, you could reassign the column using a boolean mask. `df3['overworked'] > 175` created a series of booleans, consisting of *True* for every value > 175 and *False* for every value ≤ 175. `.astype(int)` converts all *True* to 1 and all *False* to 0

```
[60]: # Define 'overworked' as working > 175 hrs/week
df2['overworked'] = (df2['overworked'] > 175).astype(int)

# Display first few rows of new column
df2['overworked'].head()
```

```
[60]: 0    0
      1    1
      2    1
      3    1
      4    0
      Name: overworked, dtype: int64
```

Drop the `average_monthly_hours` column.

```
[61]: # Drop the 'average_monthly_hours' column
df2 = df2.drop('average_monthly_hours', axis=1)

# Display first few rows of the resulting dataframe
df2.head()
```

```
[61]:   last_evaluation  number_project  tenure  work_accident  left  \
0             0.53                2      3             0      1
1             0.86                5      6             0      1
2             0.88                7      4             0      1
3             0.87                5      5             0      1
4             0.52                2      3             0      1

      promotion_last_5years  salary  department_IT  department_RandD  \
0                0          0          0          0
1                0          1          0          0
2                0          1          0          0
```

3	0	0	0	0
4	0	0	0	0

	department_accounting	department_hr	department_management	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	department_marketing	department_product_mng	department_sales	\
0	0	0	1	
1	0	0	1	
2	0	0	1	
3	0	0	1	
4	0	0	1	

	department_support	department_technical	overworked
0	0	0	0
1	0	0	1
2	0	0	1
3	0	0	1
4	0	0	0

Again, isolate the features and target variables

```
[62]: # Isolate the outcome variable
y = df2['left']

# Select the features
X = df2.drop('left', axis=1)
```

Split the data into training and testing data.

```
[63]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↳ stratify=y, random_state=0)
```

Decision tree - round 2

```
[64]: # Instantiate model
tree = DecisionTreeClassifier(random_state=0)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [4, 6, 8, None],
             'min_samples_leaf': [2, 5, 1],
             'min_samples_split': [2, 4, 6]
            }
```



```
# Assign a dictionary of scoring metrics to capture
scoring = {'accuracy', 'precision', 'recall', 'f1', 'roc_auc'}

# Instantiate GridSearch
tree2 = GridSearchCV(tree, cv_params, scoring=scoring, cv=4, refit='roc_auc')
```

```
[65]: %%time
tree2.fit(X_train, y_train)
```

```
CPU times: user 2.39 s, sys: 0 ns, total: 2.39 s
Wall time: 2.39 s
```

```
[65]: GridSearchCV(cv=4, error_score=nan,
                  estimator=DecisionTreeClassifier(ccp_alpha=0.0, class_weight=None,
                                                    criterion='gini', max_depth=None,
                                                    max_features=None,
                                                    max_leaf_nodes=None,
                                                    min_impurity_decrease=0.0,
                                                    min_impurity_split=None,
                                                    min_samples_leaf=1,
                                                    min_samples_split=2,
                                                    min_weight_fraction_leaf=0.0,
                                                    presort='deprecated',
                                                    random_state=0, splitter='best'),
                  iid='deprecated', n_jobs=None,
                  param_grid={'max_depth': [4, 6, 8, None],
                              'min_samples_leaf': [2, 5, 1],
                              'min_samples_split': [2, 4, 6]},
                  pre_dispatch='2*n_jobs', refit='roc_auc', return_train_score=False,
                  scoring={'recall', 'accuracy', 'roc_auc', 'f1', 'precision'},
                  verbose=0)
```

```
[66]: # Check the best params
tree2.best_params_
```

```
[66]: {'max_depth': 6, 'min_samples_leaf': 2, 'min_samples_split': 6}
```

```
[67]: # Check the best AUC score on CV
tree2.best_score_
```

```
[67]: 0.9586752505340426
```

This model performs very well, even without satisfaction levels and detailed hours worked data. Next, check the other scores.

```
[68]: # Get all CV scores
tree2_cv_results = make_results('decision tree2 cv', tree2, 'auc')
```

```
print(tree1_cv_results)
print(tree2_cv_results)
```

	model	precision	recall	F1	accuracy	auc
0	decision treecv	0.914552	0.916949	0.915707	0.971978	0.969819
	model	precision	recall	F1	accuracy	auc
0	decision tree2 cv	0.856693	0.903553	0.878882	0.958523	0.958675

Some of the other scores fell. That's to be expected given fewer features were taken into account in this round of the model. Still, the scores are very good.

Random forest - round 2

```
[69]: # Instantiate model
rf = RandomForestClassifier(random_state=0)

# Assign a dictionary of hyperparameters to search over
cv_params = {'max_depth': [3,5, None],
             'max_features': [1.0],
             'max_samples': [0.7, 1.0],
             'min_samples_leaf': [1,2,3],
             'min_samples_split': [2,3,4],
             'n_estimators': [300, 500],
             }

# Assign a dictionary of scoring metrics to capture
scoring = {'accuracy', 'precision', 'recall', 'f1', 'roc_auc'}

# Instantiate GridSearch
rf2 = GridSearchCV(rf, cv_params, scoring=scoring, cv=4, refit='roc_auc')
```

```
[70]: %%time
rf2.fit(X_train, y_train)
```

CPU times: user 6min 53s, sys: 0 ns, total: 6min 53s
Wall time: 6min 53s

```
[70]: GridSearchCV(cv=4, error_score=nan,
                  estimator=RandomForestClassifier(bootstrap=True, ccp_alpha=0.0,
                                                    class_weight=None,
                                                    criterion='gini', max_depth=None,
                                                    max_features='auto',
                                                    max_leaf_nodes=None,
                                                    max_samples=None,
                                                    min_impurity_decrease=0.0,
                                                    min_impurity_split=None,
                                                    min_samples_leaf=1,
                                                    min_samples_split=2,
                                                    min_weight_fraction_leaf=0.0,
```

```

n_estimators=100, n_jobs=None,...
verbose=0, warm_start=False),
iid='deprecated', n_jobs=None,
param_grid={'max_depth': [3, 5, None], 'max_features': [1.0],
            'max_samples': [0.7, 1.0],
            'min_samples_leaf': [1, 2, 3],
            'min_samples_split': [2, 3, 4],
            'n_estimators': [300, 500]},
pre_dispatch='2*n_jobs', refit='roc_auc', return_train_score=False,
scoring={'recall', 'accuracy', 'roc_auc', 'f1', 'precision'},
verbose=0)

```

```

[71]: # Write pickle
write_pickle(path, rf2, 'hr_rf2')

```

```

[72]: # Read in pickle
rf2 = read_pickle(path, 'hr_rf2')

```

```

[73]: # Check best params
rf2.best_params_

```

```

[73]: {'max_depth': 5,
      'max_features': 1.0,
      'max_samples': 0.7,
      'min_samples_leaf': 2,
      'min_samples_split': 2,
      'n_estimators': 300}

```

```

[74]: rf2.best_score_

```

```

[74]: 0.9648100662833985

```

```

[75]: # Get all CV scores
rf2_cv_results = make_results('random forest2 cv', rf2, 'auc')
print(tree2_cv_results)
print(rf2_cv_results)

```

	model	precision	recall	F1	accuracy	auc
0	decision tree2 cv	0.856693	0.903553	0.878882	0.958523	0.958675
	model	precision	recall	F1	accuracy	auc
0	random forest2 cv	0.866758	0.878754	0.872407	0.957411	0.96481

Again, the scores dropped slightly, but the random forest performs better than the decision tree if used AUC as the decision metric.

Score the champion model on the test set now.

```
[76]: # Get prediction on test data
rf2_test_scores = get_scores('random forest2 test', rf2, X_test, y_test)
rf2_test_scores
```

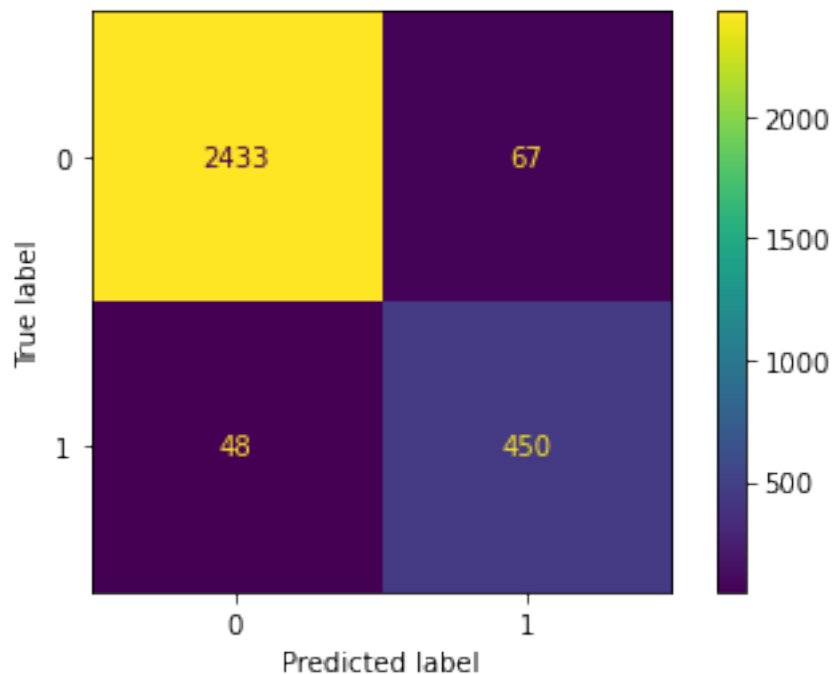
```
[76]:          model  precision    recall    f1  accuracy      AUC
0  random forest2 test   0.870406  0.903614  0.8867  0.961641  0.938407
```

This seems to be a stable, well-performing final model.

Plot a confusion matrix to visualize how well it predicts on the test set.

```
[77]: # Generate array of values for confusion matrix
preds = rf2.best_estimator_.predict(X_test)
cm = confusion_matrix(y_test, preds, labels=rf2.classes_)

# Plot confusion matrix
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=rf2.classes_)
disp.plot(values_format='');
```

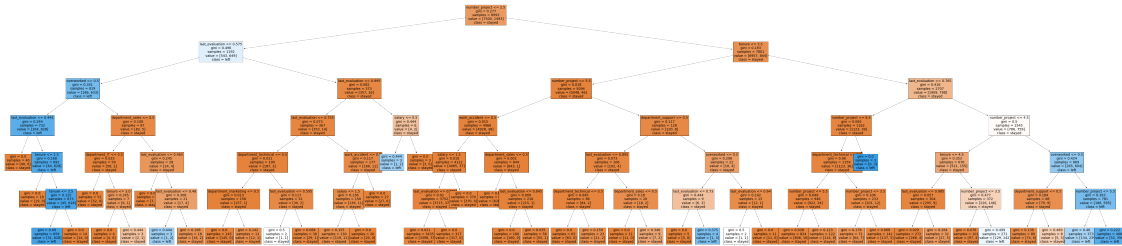


The model predicts more false positives than false negatives, which means that some employees may be identified as at risk of quitting or getting fired, when that's actually not the case. But this is still a good model.

For exploratory purposes, you might want to inspect the splits of the decision tree model and the most important features in this random forest model.

Decision tree splits

```
[78]: # Plot the tree
plt.figure(figsize=(85,20))
plot_tree(tree2.best_estimator_, max_depth=6, fontsize=14, feature_names=X.
    ↪columns,
          class_names={0: 'stayed', 1: 'left'}, filled=True);
plt.show()
```



Decision tree feature importance

```
[79]: tree2_importances = pd.DataFrame(tree2.best_estimator_.feature_importances_,
    columns=['gini_importance'],
    index=X.columns
    )

tree2_importances = tree2_importances.sort_values(by='gini_importance',
    ↪ascending=False)

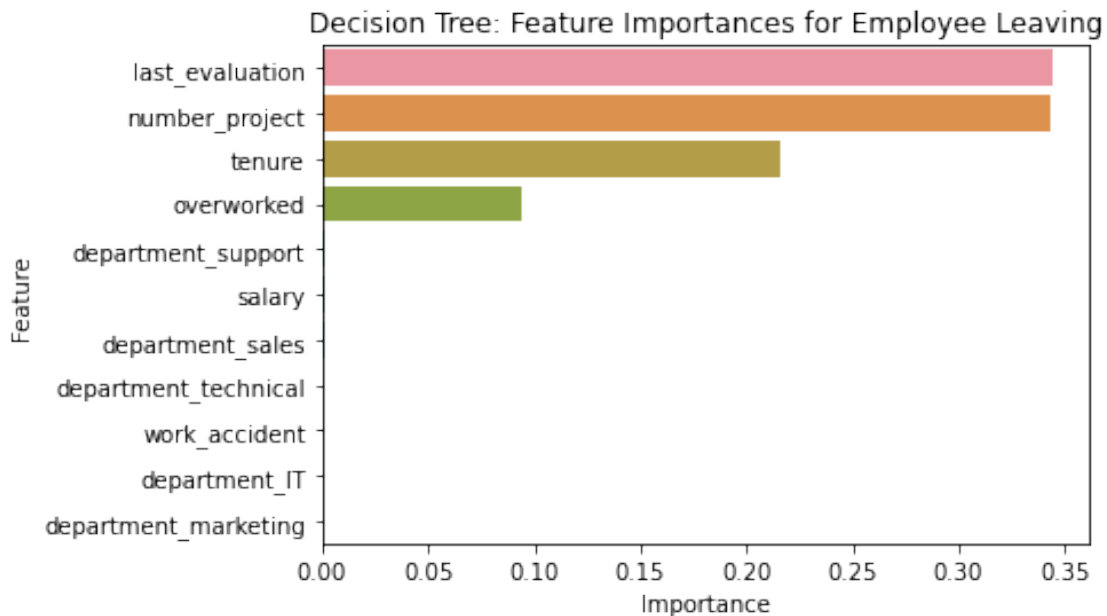
# Only extract the features with importances > 0
tree2_importances = tree2_importances[tree2_importances['gini_importance'] != 0]
tree2_importances
```

```
[79]:
```

	gini_importance
last_evaluation	0.343958
number_project	0.343385
tenure	0.215681
overworked	0.093498
department_support	0.001142
salary	0.000910
department_sales	0.000607
department_technical	0.000418
work_accident	0.000183
department_IT	0.000139
department_marketing	0.000078

Create a barplot to visualize the decision tree feature importance.

```
[80]: sns.barplot(data=tree2_importances, x='gini_importance', y=tree2_importances.
    ↪index, orient='h')
plt.title('Decision Tree: Feature Importances for Employee Leaving',
    ↪fontsize=12)
plt.ylabel('Feature')
plt.xlabel('Importance')
plt.show()
```



The barplot above shows that this decision tree model, `last_evaluation`, `number_project`, `tenure`, and `overworked` have the highest importance, in that order. These variables are most helpful in predicting the outcome variable, `left`.

Random forest feature importance

Plot the feature importances for the random forest model.

```
[83]: # Get feature importances
feat_impt = rf2.best_estimator_.feature_importances_

# Get indices of top 10 features
ind = np.argsort(rf2.best_estimator_.feature_importances_, -10)[-10:]

# Get column labels of top 10 features
feat = X.columns[ind]

# Filter `feat_impt` to consist of top 10 feature importances
feat_impt = feat_impt[ind]
```

```

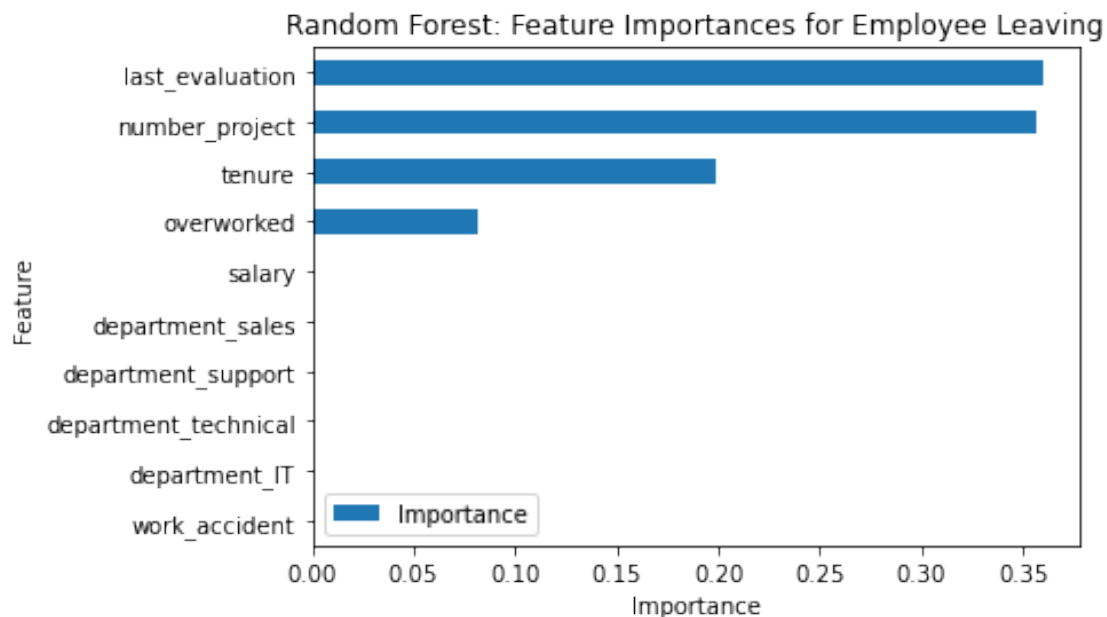
y_df = pd.DataFrame({"Feature":feat,"Importance":feat_impt})
y_sort_df = y_df.sort_values("Importance")
fig = plt.figure()
ax1 = fig.add_subplot(111)

y_sort_df.plot(kind='barh',ax=ax1,x="Feature",y="Importance")

ax1.set_title("Random Forest: Feature Importances for Employee Leaving",
↳fontsize=12)
ax1.set_ylabel("Feature")
ax1.set_xlabel("Importance")

plt.show()

```



The plot above shows that in this random forest model, **last_evaluation**, **number_object**, **tenure**, and **overworked** have the highest importance, in that order. These variables are most helpful in predicting the outcome variables, **left**, and they are the same as the ones used by the decision tree model.

5 pacE: Execute Stage

- Interpret model performance and results
- Share actionable steps with stakeholders

Recall evaluation metrics

- **AUC** is the area under the ROC curve; it's also considered the probability that the model ranks a random positive example more highly than a random negative example.
- **Precision** measures the proportion of data points predicted as True that are actually True, in other words, the proportion of positive predictions that are true positives.
- **Recall** measures the proportion of data points that are predicted as True, out of all the data points that are actually True. In other words, it measures the proportion of positives that are correctly classified.
- **Accuracy** measures the proportion of data points that are correctly classified.
- **F1-score** is an aggregation of precision and recall.

Reflect on these questions as you complete the executing stage.

- What key insights emerged from your model(s)?
- What business recommendations do you propose based on the models built?
- What potential recommendations would you make to your manager/company?
- Do you think your model could be improved? Why or why not? How?
- Given what you know about the data and the models you were using, what other questions could you address for the team?
- What resources do you find yourself using as you complete this stage? (Make sure to include the links.)
- Do you have any ethical considerations in this stage?

What key insights emerged from your model(s)?

The classification report from the logistics regression model showed that the model achieved a precision of 79%, recall of 82%, f1_score of 80%(all weighted averages), and accuracy of 82%. The decision tree model achieved an accuracy score of 96.9% indicating that the model can predict employees who will leave very well. Next, a random forest model was constructed and the results were compared with the decision tree model. The evaluation scores of the random forest were better than those of the decision tree model, with the exception of recall(the recall score of the random forest model is approximately 0.001 lower, which is a negligible amount). This indicated that the random forest mostly outperformed the decision tree model.

Due to the high evaluation scored from the model there was a suspicion of data leakage. The `average_monthly_hours` was identified as a potential source of data leakage. Therefore it was decided to apply feature engineering techniques. The `average_monthly_hours` was transformed with a boolean mask that classified whether an employee is overworked (1) or underworked (0). The `satisfaction_level` column was also dropped from the dataset. With these new features a second second round of decision tree and random forest models were deployed and the results were compared with the first round that contained the old features.

The second random forest test result showed that the model has scored an accuracy score of 96.1% on the test, with a precision of 87%, and a recall of 88.6%. This indicated that the model was stable and that it is a good final model. The model does predict more false positives than false negatives, which means that some employees may be identified as at risk of quitting or getting fired, when that's actually not the case. But this is still a good model.

What business recommendations do you propose based on the models built?

Based on insights about `number_project` and `average_monthly_hours`, it's recommended that managers review employee workload for employees with too few or too many projects and adjust them to a more suitable number. The model can be used to identify employees at high risk of leaving.

It's recommended that a system is implemented where managers and HR can proactively check in with these employees to discuss their satisfaction, career goals, and workload. Since both very low and very high last evaluation scores were indicators of attrition, it's recommended that a deeper analysis is conducted on the performance review process. High performers who are leaving the company should find out how to retain them. For low performers, the company should understand if they are disengaged or if the performance review system needs to be adjusted. The model showed that employees with high `time_spend_company` are leaving. This might be an indicator for lack of career progression. It's recommended that the company creates and communicates clear career advancement plans and mentor programs for employees at a high-risk tenure.

What potential recommendations would you make to your manager/company?

I recommend that the company establish a proactive retention team (HR, data science, and department managers) to monitor the attrition model's predictions and develop target interventions. Limitations in the data suggest that improved data collection can benefit the organisation. By collecting more detailed data on manager-employee relationships, team dynamics, and specific reasons for leaving (e.g., exit interview data) would improve future models. It's recommended that instead of a single large survey, more frequent pulse surveys are conducted. This allows the company to get real-time feedback and act quickly.

Do you think your model could be improved? Why or why not? How?

No model is perfect and it can always be improved. However there are some limitations in the current data. The current dataset is a single snapshot in time. A more robust model can be trained on time-series data to understand how employee metrics change over time. New data features like `days_since_last_raise`, `manger_tenure`, or `commute_time` can significantly improve model performance. More advanced machine learning algorithms like CatBoost or LightGBM, or exploring unsupervised learning to cluster employees into different risk groups, can also be implemented. Lastly, collecting more data, as mentioned before, could improve valuable insights.

Given what you know about the data and the models you were using, what other questions could you address for the team?

Similar data can be used to predict if employees are likely to be high performers or have characteristics of successful employees. The models can be used to find patterns in employees who have been promoted to identify a pool of high-potential talent. The models can also be used to identify characteristics of employees who stay with the company for a long time, this can be used to optimize hiring, by hiring more people with similar traits.

What resources do you find yourself using as you complete this stage? (Make sure to include the links.) Python Libraries: scikit-learn: For building machine learning models. Pandas: For data manipulation and analysis. NumPy: For numerical operations. Matplotlib and Seaborn: For data visualization. XGBoost: For the XGBoost classifier model. Online Learning & Reference: Stack Overflow: For code-related questions. Kaggle: For datasets and example notebooks. Towards Data Science: For conceptual explanations and tutorials.

Do you have any ethical considerations at this stage? There are a few ethical considerations at this stage. First, acknowledging the possibility of algorithmic bias. If the historical data contains biases, the model could learn and perpetuate this bias. Secondly, the model uses sensitive employee data, therefore data privacy should be prioritised. Being transparent with employees about what the data is used for is important. Lastly, the model should at all time be used for fairness, not to discriminate. The model should not be used to fire employees but rather help managers find ways

to support employees. Model explainability is also important, being able to explain why a model flagged an employee as high- risk is crucial for trust and effective intervention.

5.1 Step 4. Results and Evaluation

- Interpret model
- Evaluate model performance using metrics
- Prepare results, visualizations, and actionable steps to share with stakeholders

5.1.1 Summary of model results

Logistic Regression

The logistic regression model achieved precision of 80%, recall of 83%, f1-score of 80% (all weighted averages), and accuracy of 83%, on the test set.

Tree-based Machine Learning

After conducting feature engineering, the decision tree model achieved AUC of 93.8%, precision of 87.0%, recall of 90.4%, f1-score of 88.7%, and accuracy of 96.2%, on the test set. The random forest modestly outperformed the decision tree model.

5.1.2 Conclusion, Recommendations, Next Steps

The models and the feature importances extracted from the models confirm that employees at the company are overworked.

To retain employees, the following recommendations could be presented to the stakeholders:

- Cap the number of projects that employees can work on.
- Consider promoting employees who have been with the company for atleast four years, or conduct further investigation about why four-year tenured employees are so dissatisfied.
- Either reward employees for working longer hours, or don't require them to do so.
- If employees aren't familiar with the company's overtime pay policies, inform them about this. If the expectations around workload and time off aren't explicit, make them clear.
- Hold company-wide and within-team discussions to understand and address the company work culture, across the board and in specific contexts.
- High evaluation scores should not be reserved for employees who work 200+ hours per month. Consider a proportionate scale for rewarding employees who contribute more/put in more effort.

Next Steps

It may be justified to still have some concern about data leakage. It could be prudent to consider how predictions change when `last_evaluation` is removed from the data. It's possible that evaluations aren't performed very frequently, in which case it would be useful to be able to predict employee retention without this feature. It's also possible that the evaluation score determines whether an employee leaves or stays, in which case it could be useful to pivot and try to predict performance score. The same could be said for satisfaction score.

For another project, you could try building a K-means model on this data and analyzing the clusters. This may yield valuable insight.

Congratulations! You've completed this lab. However, you may not notice a green check mark next to this item on Coursera's platform. Please continue your progress regardless of the check mark. Just click on the "save" icon at the top of this notebook to ensure your work has been logged.